



TAB 1 · OBSERVE

```
$ kubectl get deployments -w
$ kubectl get pods -w
$ kubectl get rs -w
```

TAB 2 · CONTROL

```
$ kubectl create deploy web --image=nginx --replicas=2
$ kubectl delete pod <pod-name>
$ kubectl scale deploy web --replicas=4
```

STEP 1 · Create

RUN

```
$ kubectl create deployment web --image=nginx --replicas=2
deployment.apps/web created
```

EXPECTED in TAB 1

```
web 2/2 READY
web-aaaaa 1/1 Running
web-bbbbb 1/1 Running
```

STEP 2 · Kill a Pod

RUN

```
$ kubectl delete pod web-aaaaa
pod "web-aaaaa" deleted
```

EXPECTED

```
web-aaaaa Terminating
web-cccc  ContainerCreating
web-cccc  Running # replacement!
```

STEP 3 · What we proved

A ReplicaSet owns the Pods → deletion is a mismatch, not a tragedy. The **name** died; the **promise** didn't. (Standalone pods: `kubectl run p1 --image=nginx` then delete it — silence. Try it.)

[↑ back to top](#)



STEP 1 · Scale to 5

⚙️ RUN

```
$ kubectl scale deploy web --replicas=5  
deployment.apps/web scaled
```

👁️ EXPECTED

```
three new pods  
ContainerCreating → Running  
total: 5/5
```

STEP 2 · Delete two at once

⚙️ RUN

```
$ kubectl delete pod web-cccc web-dddd
```

👁️ EXPECTED

```
# two replacements appear  
# concurrently, new random names  
back to 5/5 in seconds
```

STEP 3 · The edge — starve it

⚙️ RUN

```
$ kubectl scale deploy web --replicas=50  
$ kubectl get pods
```

👁️ YOU'LL SEE

```
many pods stuck  
Pending # not enough cluster CPU
```

The lesson: self-healing repairs *state*, not *capacity*. Scale back down: `kubectl scale deploy web --replicas=2`.



Scale is just editing the promise. The ReplicaSet reconciles upward and downward — surplus Pods get terminated, deficits get filled. Same loop, both directions.

[↑ back to top](#)



STEP 1 · Scale to 5

⚙️ RUN

```
$ kubectl scale deploy web --replicas=5  
deployment.apps/web scaled
```

👁️ EXPECTED

```
three new pods  
ContainerCreating → Running  
total: 5/5
```

STEP 2 · Delete two at once

⚙️ RUN

```
$ kubectl delete pod web-cccc web-dddd
```

👁️ EXPECTED

```
# two replacements appear  
# concurrently, new random names  
back to 5/5 in seconds
```

STEP 3 · The edge — starve it

⚙️ RUN

```
$ kubectl scale deploy web --replicas=50  
$ kubectl get pods
```

👁️ YOU'LL SEE

```
many pods stuck  
Pending # not enough cluster CPU
```

The lesson: self-healing repairs *state*, not *capacity*. Scale back down: `kubectl scale deploy web --replicas=2`.



Scale is just editing the promise. The ReplicaSet reconciles upward and downward — surplus Pods get terminated, deficits get filled. Same loop, both directions.

[↑ back to top](#)



STEP 1 · Same name, two apartments

⚙️ RUN

```
$ kubectl create ns dev
$ kubectl run nginx --image=nginx -n dev
$ kubectl run nginx --image=nginx
```

👁️ EXPECTED

```
$ kubectl get pods -A | grep nginx
default    nginx    Running
dev        nginx    Running
# two different pods, zero collision
```

STEP 2 · Namespace flags matter

⚙️ RUN

```
$ kubectl get pods          # default only
$ kubectl get pods -n dev   # dev only
$ kubectl get pods -A      # everything
```

👁️ NOTE

```
$ kubectl get nodes -n dev
# same node list – -n is IGNORED
# nodes are cluster-scoped!
```

STEP 3 · The cascade ⚠️

⚙️ RUN

```
$ kubectl delete ns dev
namespace "dev" deleted
```

👁️ EXPECTED

```
# everything inside dev is
# terminated and removed –
# the pod included
```



Deleting a namespace deletes EVERYTHING in it. There is no trash can, no undo. In production, some teams even pin RBAC so only admins can delete namespaces.

[↑ back to top](#)



STEP 1 · Same name, two apartments

⚙️ RUN

```
$ kubectl create ns dev
$ kubectl run nginx --image=nginx -n dev
$ kubectl run nginx --image=nginx
```

👁️ EXPECTED

```
$ kubectl get pods -A | grep nginx
default    nginx    Running
dev        nginx    Running
# two different pods, zero collision
```

STEP 2 · Namespace flags matter

⚙️ RUN

```
$ kubectl get pods          # default only
$ kubectl get pods -n dev   # dev only
$ kubectl get pods -A       # everything
```

👁️ NOTE

```
$ kubectl get nodes -n dev
# same node list – -n is IGNORED
# nodes are cluster-scoped!
```

STEP 3 · The cascade ⚠️

⚙️ RUN

```
$ kubectl delete ns dev
namespace "dev" deleted
```

👁️ EXPECTED

```
# everything inside dev is
# terminated and removed –
# the pod included
```



Deleting a namespace deletes EVERYTHING in it. There is no trash can, no undo. In production, some teams even pin RBAC so only admins can delete namespaces.

[↑ back to top](#)



STEP 1 · Upgrade

⚙️ RUN

```
$ kubectl set image deploy web nginx=nginx:1.26
$ kubectl rollout status deploy web
```

👁️ EXPECTED

```
Waiting for deployment...
successfully rolled out
$ kubectl get rs
# two RS: old 0, new 2
```

STEP 2 · Ship a broken tag

⚙️ RUN

```
$ kubectl set image deploy web nginx=nginx:1.99
$ kubectl rollout status deploy web
```

👁️ EXPECTED

```
Waiting... 1 unavailable
$ kubectl get pods
one pod in ImagePullBackOff
OLD pods still running!
```

STEP 3 · Roll back

⚙️ RUN

```
$ kubectl rollout undo deploy web
$ kubectl rollout history deploy web
```

👁️ EXPECTED

```
successfully rolled out
# bad pod terminated, good ones back
# history shows a NEW revision (v3)
```



The safety net is structural: a bad rollout can never take down the old version — the Deployment only removes old Pods once new ones are Ready. Broken tag? The cluster waits, old version serves, you type undo.

[↑ back to top](#)



STEP 1 · Upgrade

⚙️ RUN

```
$ kubectl set image deploy web nginx=nginx:1.26
$ kubectl rollout status deploy web
```

👁️ EXPECTED

```
Waiting for deployment...
successfully rolled out
$ kubectl get rs
# two RS: old 0, new 2
```

STEP 2 · Ship a broken tag

⚙️ RUN

```
$ kubectl set image deploy web nginx=nginx:1.99
$ kubectl rollout status deploy web
```

👁️ EXPECTED

```
Waiting... 1 unavailable
$ kubectl get pods
one pod in ImagePullBackOff
OLD pods still running!
```

STEP 3 · Roll back

⚙️ RUN

```
$ kubectl rollout undo deploy web
$ kubectl rollout history deploy web
```

👁️ EXPECTED

```
successfully rolled out
# bad pod terminated, good ones back
# history shows a NEW revision (v3)
```



The safety net is structural: a bad rollout can never take down the old version — the Deployment only removes old Pods once new ones are Ready. Broken tag? The cluster waits, old version serves, you type undo.

[↑ back to top](#)

**STEP 1 · Taint AFTER the Pod exists**

RUN

```
$ kubectl taint node <node> env=prod:NoSchedule
$ kubectl scale deploy web --replicas=4
```

EXPECTED

```
# existing pods keep running
# NEW 4th pod → Pending
0/1 Nodes available
...taint...that pod tolerates not
```

Key insight: **NoSchedule** only gates *new* placements — it never evicts.

STEP 2 · Rescue with a toleration

```
spec:
  tolerations:
  - key: env
    operator: Equal
    value: prod
    effect: NoSchedule # pod may now LAND on the tainted node
```

Apply → the Pending pod schedules immediately. **Taint = the lock. Toleration = the key.**

STEP 3 · NoExecute — the eviction

RUN

```
$ kubectl taint node <node> env=prod:NoExecute
```

EXPECTED

```
# non-tolerating pods on that
# node get EVICTED — now
```

Effect	Existing Pods	New Pods
NoSchedule	stay	blocked (unless tolerated)
PreferNoSchedule	stay	avoided if possible
NoExecute	evicted	blocked (unless tolerated)



Remember the direction: taints repel (node → pods); tolerations permit (pod → tainted node). Taints **never attract** — for that you'd use nodeSelector/affinity.

[↑ back to top](#)



STEP 1 · Taint AFTER the Pod exists

⚙️ RUN

```
$ kubectl taint node <node> env=prod:NoSchedule
$ kubectl scale deploy web --replicas=4
```

👁️ EXPECTED

```
# existing pods keep running
# NEW 4th pod → Pending
0/1 Nodes available
...taint...that pod tolerates not
```

Key insight: **NoSchedule** only gates *new* placements — it never evicts.

STEP 2 · Rescue with a toleration

```
spec:
  tolerations:
  - key: env
    operator: Equal
    value: prod
    effect: NoSchedule # pod may now LAND on the tainted node
```

Apply → the Pending pod schedules immediately. **Taint = the lock. Toleration = the key.**

STEP 3 · NoExecute — the eviction

⚙️ RUN

```
$ kubectl taint node <node> env=prod:NoExecute
```

👁️ EXPECTED

```
# non-tolerating pods on that
# node get EVICTED — now
```

Effect	Existing Pods	New Pods
NoSchedule	stay	blocked (unless tolerated)
PreferNoSchedule	stay	avoided if possible
NoExecute	evicted	blocked (unless tolerated)



Remember the direction: taints repel (node → pods); tolerations permit (pod → tainted node). Taints **never attract** — for that you'd use nodeSelector/affinity.

[↑ back to top](#)



Q1. A Kubernetes cluster consists of...

A. API Server + etcd only **B.** Control Plane + Worker Nodes **C.** Nodes + Pods only **D.** Masters only

Q2. Which component is the ONLY door to the cluster — everyone talks through it?

A. etcd **B.** Scheduler **C.** API Server **D.** kubelet

Q3. A worker dies running 1 of 3 Deployment replicas. What happens?

A. Nothing until you fix the node **B.** Replica count drops to 2 **C.** A replacement Pod is created on a healthy node **D.** The whole Deployment restarts

Q4. With 5 Control-Plane nodes, how many can fail before etcd loses quorum?

A. 1 **B.** 3 **C.** 2 **D.** 4

Q5. `kubectl get nodes -n dev` returns...

A. dev's nodes **B.** an error **C.** nothing **D.** all nodes — nodes are cluster-scoped, the flag is ignored

Q6. You delete a ReplicaSet-owned Pod. Result?

A. Gone forever **B.** Recreated with a new name **C.** Same Pod restarts **D.** ReplicaSet is deleted too

Q7. Pod stuck in `ImagePullBackOff`. Most likely cause?

A. Bad/typo image tag or missing registry credentials **B.** App crashed **C.** No CPU available **D.** Service misconfigured

Q8. Valid NodePort range?

A. 1024–65535 **B.** 1–1024 **C.** 8080–9999 **D.** 30000–32767

Q9. kube-proxy's actual job?

A. Forward packets between nodes **B.** Assign Pod IPs **C.** Watch Services/Endpoints and program node routing rules **D.** Run CoreDNS

Q10. Which taint effect **evicts existing** non-tolerating Pods?

A. NoExecute **B.** NoSchedule **C.** PreferNoSchedule **D.** EvictOnly

[↑ back to top](#)



Q1. A Kubernetes cluster consists of...

A. API Server + etcd only **B.** Control Plane + Worker Nodes **C.** Nodes + Pods only **D.** Masters only

Q2. Which component is the ONLY door to the cluster — everyone talks through it?

A. etcd **B.** Scheduler **C.** API Server **D.** kubelet

Q3. A worker dies running 1 of 3 Deployment replicas. What happens?

A. Nothing until you fix the node **B.** Replica count drops to 2 **C.** A replacement Pod is created on a healthy node **D.** The whole Deployment restarts

Q4. With 5 Control-Plane nodes, how many can fail before etcd loses quorum?

A. 1 **B.** 3 **C.** 2 **D.** 4

Q5. `kubectl get nodes -n dev` returns...

A. dev's nodes **B.** an error **C.** nothing **D.** all nodes — nodes are cluster-scoped, the flag is ignored

Q6. You delete a ReplicaSet-owned Pod. Result?

A. Gone forever **B.** Recreated with a new name **C.** Same Pod restarts **D.** ReplicaSet is deleted too

Q7. Pod stuck in `ImagePullBackOff`. Most likely cause?

A. Bad/typo image tag or missing registry credentials **B.** App crashed **C.** No CPU available **D.** Service misconfigured

Q8. Valid NodePort range?

A. 1024–65535 **B.** 1–1024 **C.** 8080–9999 **D.** 30000–32767

Q9. kube-proxy's actual job?

A. Forward packets between nodes **B.** Assign Pod IPs **C.** Watch Services/Endpoints and program node routing rules **D.** Run CoreDNS

Q10. Which taint effect **evicts existing** non-tolerating Pods?

A. NoExecute **B.** NoSchedule **C.** PreferNoSchedule **D.** EvictOnly

[↑ back to top](#)



Q	Ans	Why — one line
1	B	Cluster = Control Plane (decides) + Workers (execute) — CH 01
2	C	API Server is the only door; only it touches etcd — CH 02/04
3	C	Desired 3 $\neq$ Actual 2 $\rightarrow$ reconciliation creates a replacement — CH 06/16
4	C	Quorum of 5 = 3 $\rightarrow$ tolerate $\text{floor}(5/2) = 2$ failures — CH 08
5	D	Nodes are cluster-scoped; -n is silently ignored — CH 11
6	B	Ownership means the promise survives; new name each time — CH 15/16
7	A	Pull failed: bad tag or private registry without credentials — CH 23
8	D	30000–32767, auto-assigned if omitted — CH 25
9	C	Watch & program iptables/IPVS — never forwards traffic itself — CH 27
10	A	NoExecute evicts now; NoSchedule/Prefer only block placement — CH 33

**9–10**

Exam-ready on this material

**6–8**Re-read the missed chapters' 
boxes**<6**

Re-run the labs — then retake



Missed Q4 or Q10? Those are the two most common interview traps: quorum math and NoExecute eviction. Missed Q5? The cluster-scoped list (Nodes, PVs, ClusterRoles, Namespaces) is a free exam point — memorize it.

[↑ back to top](#)



Q	Ans	Why — one line
1	B	Cluster = Control Plane (decides) + Workers (execute) — CH 01
2	C	API Server is the only door; only it touches etcd — CH 02/04
3	C	Desired 3 $\neq$ Actual 2 $\rightarrow$ reconciliation creates a replacement — CH 06/16
4	C	Quorum of 5 = 3 $\rightarrow$ tolerate $\text{floor}(5/2) = 2$ failures — CH 08
5	D	Nodes are cluster-scoped; -n is silently ignored — CH 11
6	B	Ownership means the promise survives; new name each time — CH 15/16
7	A	Pull failed: bad tag or private registry without credentials — CH 23
8	D	30000–32767, auto-assigned if omitted — CH 25
9	C	Watch & program iptables/IPVS — never forwards traffic itself — CH 27
10	A	NoExecute evicts now; NoSchedule/Prefer only block placement — CH 33

**9–10**

Exam-ready on this material

**6–8**Re-read the missed chapters' 
boxes**<6**

Re-run the labs — then retake



Missed Q4 or Q10? Those are the two most common interview traps: quorum math and NoExecute eviction. Missed Q5? The cluster-scoped list (Nodes, PVs, ClusterRoles, Namespaces) is a free exam point — memorize it.

[↑ back to top](#)

Cluster — Control Plane + Worker Nodes managed as one unit.

Control Plane — the brain: API Server, etcd, Scheduler, Controllers. Makes decisions; runs no user workloads (by taint).

API Server — the only door 🚪. All reads/writes (kubectl, kubelet, controllers) go through it; only it touches etcd.

etcd — the cluster's memory 🗄️. Needs quorum (majority) to write — why production runs 3 or 5 Control Planes.

Scheduler — assigns new Pods to Nodes based on resources, taints, affinity.

Controller — reconciliation loop: watch actual vs desired, act to close the gap.

kubelet — the node agent 🧑. Talks to the API Server, runs Pods via the runtime, reports status.

Container Runtime — pulls images and runs containers (containerd, CRI-O).

kube-proxy — watches Services/Endpoints, programs iptables/IPVS rules per node. Never forwards packets itself.

Pod — smallest deployable unit: 1+ containers sharing IP, volumes, lifecycle.

ReplicaSet — the guarantor: keeps N matching-labeled Pods alive via a template.

Deployment — manages ReplicaSets to enable versioned, zero-downtime rollouts and rollbacks.

Namespace — logical partition of names + RBAC + quotas inside one cluster. Apartments, not buildings.

Service — stable virtual IP + DNS name in front of a changing set of Pods.

NodePort — Service type opening one high port (30000–32767) on every node.

Taint / Toleration — node-side repulsion / pod-side permission. NoSchedule blocks placement; NoExecute also evicts.



Self-test the glossary: cover the right side, read each term, say the one-line definition aloud. If you can't — that's tonight's reading.

↑ back to top



Cluster — Control Plane + Worker Nodes managed as one unit.

Control Plane — the brain: API Server, etcd, Scheduler, Controllers. Makes decisions; runs no user workloads (by taint).

API Server — the only door 🚪. All reads/writes (kubectl, kubelet, controllers) go through it; only it touches etcd.

etcd — the cluster's memory 🗄️. Needs quorum (majority) to write — why production runs 3 or 5 Control Planes.

Scheduler — assigns new Pods to Nodes based on resources, taints, affinity.

Controller — reconciliation loop: watch actual vs desired, act to close the gap.

kubelet — the node agent 🧑. Talks to the API Server, runs Pods via the runtime, reports status.

Container Runtime — pulls images and runs containers (containerd, CRI-O).

kube-proxy — watches Services/Endpoints, programs iptables/IPVS rules per node. Never forwards packets itself.

Pod — smallest deployable unit: 1+ containers sharing IP, volumes, lifecycle.

ReplicaSet — the guarantor: keeps N matching-labeled Pods alive via a template.

Deployment — manages ReplicaSets to enable versioned, zero-downtime rollouts and rollbacks.

Namespace — logical partition of names + RBAC + quotas inside one cluster. Apartments, not buildings.

Service — stable virtual IP + DNS name in front of a changing set of Pods.

NodePort — Service type opening one high port (30000–32767) on every node.

Taint / Toleration — node-side repulsion / pod-side permission. NoSchedule blocks placement; NoExecute also evicts.



Self-test the glossary: cover the right side, read each term, say the one-line definition aloud. If you can't — that's tonight's reading.

↑ back to top



LIFECYCLE

```
$ kubectl create deploy web --image=nginx --replicas=3
$ kubectl get pods -w
$ kubectl scale deploy web --replicas=5
$ kubectl set image deploy/web nginx=nginx:1.26
$ kubectl rollout status|history|undo deploy/web
$ kubectl delete deploy web
```

DEBUG

```
$ kubectl describe pod P # events at BOTTOM
$ kubectl logs P --previous # crashed container
$ kubectl exec -it P -- sh
$ kubectl get events --sort-by=.lastTimestamp
Pending→scheduler · ImagePull→registry
CrashLoop→app
```

SERVICE TYPES

Type	Reach
ClusterIP	inside only (VIP + DNS)
NodePort	node:30000–32767 from outside
LoadBalancer	cloud LB → NodePorts

TAINTS

```
$ kubectl taint node N k=v:NoSchedule
$ kubectl taint node N k=v:NoExecute
$ kubectl taint node N k=v:NoSchedule- # remove
tolerations: key/op/value/effect
```

FACTS WORTH A JOB OFFER

Cluster = CP + Workers · API Server is the only door · only it touches etcd

Quorum: 3 CP → tolerate 1 · **2 CP is the worst number** · odd counts only

Namespace partitions **names** , not hardware · Nodes are cluster-scoped

Standalone Pod dies = gone · owned Pod dies = replaced (new name)

Deployment → ReplicaSet(s) → Pods · rollback = new revision, not a rewind

ClusterIP is a fiction · kube-proxy programs rules, forwards nothing

One Pod = one IP · containers share localhost, volumes, and fate

[↑ back to top](#)



LIFECYCLE

```
$ kubectl create deploy web --image=nginx --replicas=3
$ kubectl get pods -w
$ kubectl scale deploy web --replicas=5
$ kubectl set image deploy/web nginx=nginx:1.26
$ kubectl rollout status|history|undo deploy/web
$ kubectl delete deploy web
```

DEBUG

```
$ kubectl describe pod P # events at BOTTOM
$ kubectl logs P --previous # crashed container
$ kubectl exec -it P -- sh
$ kubectl get events --sort-by=.lastTimestamp
Pending→scheduler · ImagePull→registry
CrashLoop→app
```

SERVICE TYPES

Type	Reach
ClusterIP	inside only (VIP + DNS)
NodePort	node:30000–32767 from outside
LoadBalancer	cloud LB → NodePorts

TAINTS

```
$ kubectl taint node N k=v:NoSchedule
$ kubectl taint node N k=v:NoExecute
$ kubectl taint node N k=v:NoSchedule- # remove
tolerations: key/op/value/effect
```

FACTS WORTH A JOB OFFER

Cluster = CP + Workers · API Server is the only door · only it touches etcd

Quorum: 3 CP → tolerate 1 · **2 CP is the worst number** · odd counts only

Namespace partitions **names** , not hardware · Nodes are cluster-scoped

Standalone Pod dies = gone · owned Pod dies = replaced (new name)

Deployment → ReplicaSet(s) → Pods · rollback = new revision, not a rewind

ClusterIP is a fiction · kube-proxy programs rules, forwards nothing

One Pod = one IP · containers share localhost, volumes, and fate

[↑ back to top](#)



The whole book in one tree. Close this page, draw it on paper, then check yourself.

```

🔗 CLUSTER
├── 🗨️ CONTROL PLANE x3 (quorum)
│   ├── API Server 🖨️ ← kubectl · kubelet · controllers · scheduler
│   ├── etcd 🗄️ ← API Server only
│   ├── Scheduler ⚙️ Pod → Node placement
│   └── Controllers 🗨️ desired ↔ actual, forever
├── 📁 NAMESPACES frontend · ai · dev (+ 4 system ns)
│   ├── Deployments → ReplicaSets → Pods 🍃 (+ Services · DNS)
└── 👤 WORKERS
    ├── kubelet 👤 agent ↔ API Server
    ├── Runtime ⚙️ containers
    └── kube-proxy 🌐 VIP rules · DNAT · never forwards
  
```

✅ You can now...

Draw the two halves and name all 7 components with one verb each

Explain self-healing as a **reconciliation loop**, not magic

Do the quorum math and say why 2 Control Planes are the worst number

Trace `nginx:1.25` → `1.26` → broken → undo through the ReplicaSet versions

Narrate one packet: DNS → VIP → DNAT → Pod → conntrack → home

Taint a node, watch a Pod go Pending, write the toleration that rescues it



Where to next: Secrets · ConfigMaps & RBAC (Book 02) · Services deep-dive & Ingress (Book 03) · Taints & Tolerations in full (Book 04). The spine you just learned is the same spine in every one of them.

— End of Book · 🎬 —



The whole book in one tree. Close this page, draw it on paper, then check yourself.

```

🔗 CLUSTER
├── 🗨️ CONTROL PLANE x3 (quorum)
│   ├── API Server 🖨️ ← kubectl · kubelet · controllers · scheduler
│   ├── etcd 🗄️ ← API Server only
│   ├── Scheduler 🎯 Pod → Node placement
│   └── Controllers 🗨️ desired ↔ actual, forever
├── 📁 NAMESPACES frontend · ai · dev (+ 4 system ns)
│   ├── Deployments → ReplicaSets → Pods 🍃 (+ Services · DNS)
└── 👤 WORKERS
    ├── kubelet 👤 agent ↔ API Server
    ├── Runtime ⚙️ containers
    └── kube-proxy 🌐 VIP rules · DNAT · never forwards
  
```

✓ You can now...

Draw the two halves and name all 7 components with one verb each

Explain self-healing as a **reconciliation loop**, not magic

Do the quorum math and say why 2 Control Planes are the worst number

Trace `nginx:1.25` → `1.26` → broken → undo through the ReplicaSet versions

Narrate one packet: DNS → VIP → DNAT → Pod → conntrack → home

Taint a node, watch a Pod go Pending, write the toleration that rescues it



Where to next: Secrets · ConfigMaps & RBAC (Book 02) · Services deep-dive & Ingress (Book 03) · Taints & Tolerations in full (Book 04). The spine you just learned is the same spine in every one of them.

— End of Book · 🎬 —